

WCS Implementation Pack

Operations manual + Swift Vapor starter sources

Version 1.0 · May 2026

World Class Scholars · Swift-first agentic operations

Executive operating model

World Class Scholars adopts a **Swift-first, supervised agentic platform**: bounded automation, explicit workflow state, policy-gated sensitive actions, and full audit trails. Agents coordinate through a central orchestrator; humans approve identity verification, certificate issuance, and external publishing.

Core principles (from case-study literature)

- 1. Coordination must be **constrained**, not merely automated.
- 2. Every agent has defined scope, fallback path, and audit trail.
- 3. Sensitive actions (identity, issuance, policy) remain gated until trust and performance thresholds are met.
- 4. Production systems use explicit state, bounded action spaces, and supervised transitions.

Swift-first architecture recommendation

Layer	Technology	Role
iOS client	SwiftUI + SwiftData	Participant & operator UX
Domain API	Vapor 4 + Fluent + async/await	System of record, workflows
Orchestration	Node/TypeScript (platform/)	Sessions, budgets, handoffs
Workers	Python (platform/agents/)	OCR, classification, reasoning
Policy	OPA/Rego	Pre/post invocation
Data	PostgreSQL + encrypted object store	Participants, audit, documents

Server-side Swift (Vapor) owns canonical business logic; the orchestrator owns session graphs and does not persist PII long-term.

Production readiness checklist

#	Item	Owner	Status
1	DPIA for onboarding + ID document	Legal	■
2	OPA policies in CI with deny tests	Engineering	■
3	Kill-switch tested in staging	Ops	■
4	Dual-control certificate issuance	OpenEng	■
5	Audit log retention policy (≥ 1 year)	Security	■
6	On-call runbook + incident playbook	Ops	■
7	SAST/DAST in release pipeline	DevSecOps	■
8	Token budgets per workflow config	Engineering	■
9	Human reviewer SLA defined (< 4 hrs)	Program Mgmt	■
10	Pilot KPIs baselined (time-to-ready)	Program Mgmt	■
11	Red-team policy bypass exercise	Security	■
12	Rollback tested (model + policy bundle)	Engineering	■

SaaS scaling roadmap

Phase	Scope	Deliverable
-------	-------	-------------

1	WCS pilot	Onboarding + approval UI
2	Supervised production	Certificate prep, support triage
3	Multi-tenant SaaS	Tenant isolation, KMS per tenant, config packs
4	Partner orgs	Self-serve sandbox, usage billing, SOC2 path

Generated: 2026-05-16

Swift implementation summary

WCS Agentic SaaS Starter

Swift-first backend using Vapor for World Class Scholars.

Run locally

```
cd implementation-pack
docker compose up --build
```

Endpoints

- POST /participants — JSON { "email", "fullName" }; creates participant and onboarding workflow run
- POST /identity/upload — JSON { "participantID", "documentURL" }
- POST /workflows/approve — JSON { "workflowID", "approvedBy" }
- GET /health

Native run (requires Postgres)

```
export DB_HOST=127.0.0.1
export DB_USER=postgres
export DB_PASSWORD=postgres
export DB_NAME=wcs
swift run App
```

Docker image

The image uses swift:6.0-jammy and starts the server with:

```
App serve --hostname 0.0.0.0 --port 8080
```

Note on `Package.swift` platforms

platforms: [.macOS(.v13)] sets the **minimum macOS** for Apple toolchains. If docker compose build fails on Linux with a platform error, remove the platforms line for Linux-only CI images.

Swift / Vapor implementation pack

Starter SaaS backend (Vapor 4, Fluent, PostgreSQL). Copy into implementation-pack/.

implementation-pack/Package.swift

```
// swift-tools-version: 6.0
import PackageDescription

let package = Package(
    name: "WCSAgenticSaaS",
    platforms: [
        .macOS(.v13),
    ],
    products: [
        .executable(name: "App", targets: ["App"]),
    ],
    dependencies: [
        .package(url: "https://github.com/vapor/vapor.git", from: "4.99.0"),
        .package(url: "https://github.com/vapor/fluent.git", from: "4.10.0"),
        .package(url: "https://github.com/vapor/fluent-postgres-driver.git", from: "2.11.0"),
        .package(url: "https://github.com/vapor/fluent-sqlite-driver.git", from: "4.0.0"),
        .package(url: "https://github.com/vapor/jwt.git", from: "4.2.0"),
    ],
    targets: [
        .executableTarget(
            name: "App",
            dependencies: [
                .product(name: "Vapor", package: "vapor"),
                .product(name: "Fluent", package: "fluent"),
                .product(name: "FluentPostgresDriver", package: "fluent-postgres-driver"),
                .product(name: "FluentSQLiteDriver", package: "fluent-sqlite-driver"),
                .product(name: "JWT", package: "jwt"),
            ]
        ),
        .testTarget(
            name: "AppTests",
            dependencies: [
                .target(name: "App"),
                .product(name: "XCTVapor", package: "vapor"),
            ]
        ),
    ]
)
```

implementation-pack/Dockerfile

```
FROM swift:6.0-jammy
WORKDIR /app
COPY . .
RUN swift build -c release
CMD [".build/release/App", "serve", "--hostname", "0.0.0.0", "--port", "8080"]
```

implementation-pack/docker-compose.yml

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: wcs
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d wcs"]
      interval: 3s
      timeout: 3s
      retries: 10

  app:
```

```

build: .
depends_on:
  db:
    condition: service_healthy
environment:
  DB_HOST: db
  DB_PORT: "5432"
  DB_USER: postgres
  DB_PASSWORD: postgres
  DB_NAME: wcs
  JWT_SECRET: change-me-in-production
ports:
  - "8080:8080"

```

implementation-pack/README.md

WCS Agentic SaaS Starter

Swift-first backend using Vapor for World Class Scholars.

Run locally

```

```bash
cd implementation-pack
docker compose up --build
```

```

Endpoints

```

- `POST /participants` - JSON `{ "email", "fullName" }`; creates participant and onboarding workflow run
- `POST /identity/upload` - JSON `{ "participantID", "documentURL" }`
- `POST /workflows/approve` - JSON `{ "workflowID", "approvedBy" }`
- `GET /health`

```

Native run (requires Postgres)

```

```bash
export DB_HOST=127.0.0.1
export DB_USER=postgres
export DB_PASSWORD=postgres
export DB_NAME=wcs
swift run App
```

```

Docker image

The image uses `swift:6.0-jammy` and starts the server with:

```
`App serve --hostname 0.0.0.0 --port 8080`
```

Note on `Package.swift` platforms

`platforms: [.macOS(.v13)]` sets the **minimum macOS** for Apple toolchains. If `docker compose build` fails on Linux with a platform error, remove the `platforms` line for Linux-only CI images.

implementation-pack/Sources/App/entrypoint.swift

```

import Logging
import Vapor

@main
struct Entrypoint {
    static func main() async throws {
        var env = try Environment.detect()
        try LoggingSystem.bootstrap(from: &env)

        let app = try await Application.make(env)
        do {
            try configure(app)
            try await app.autoMigrate()
            try await app.execute()
            try await app.asyncShutdown()
        } catch {
            try? await app.asyncShutdown()
            throw error
        }
    }
}

```

```

    }
  }
}

```

implementation-pack/Sources/App/configure.swift

```

import Fluent
import FluentPostgresDriver
import JWT
import Vapor

func configure(_ app: Application) throws {
    app.http.server.configuration.hostname = "0.0.0.0"
    app.http.server.configuration.port = Environment.get("PORT").flatMap { Int($0) } ?? 8080

    let dbHost = Environment.get("DB_HOST") ?? "localhost"
    let dbPort = Environment.get("DB_PORT").flatMap { Int($0) } ?? SQLPostgresConfiguration.ianaPortNumber
    let dbUser = Environment.get("DB_USER") ?? "postgres"
    let dbPassword = Environment.get("DB_PASSWORD") ?? "postgres"
    let dbName = Environment.get("DB_NAME") ?? "wcs"

    let sqlConfig = SQLPostgresConfiguration(
        hostname: dbHost,
        port: dbPort,
        username: dbUser,
        password: dbPassword,
        database: dbName,
        tls: .disable
    )

    app.databases.use(DatabaseConfigurationFactory.postgres(configuration: sqlConfig), as: .psql)

    app.migrations.add(CreateParticipantMigration())
    app.migrations.add(CreateWorkflowRunMigration())

    let jwtSecret = Environment.get("JWT_SECRET") ?? "development-only-change-me"
    app.jwt.signers.use(.hs256(key: jwtSecret), kid: "wcs")

    try routes(app)
}

```

implementation-pack/Sources/App/routes.swift

```

import Fluent
import Vapor

func routes(_ app: Application) throws {
    app.post("participants") { req async throws -> Participant in
        let input = try req.content.decode(CreateParticipant.self)
        let p = Participant(email: input.email, fullName: input.fullName)
        try await p.save(on: req.db)
        let run = WorkflowRun(
            workflowType: "onboarding",
            payload: "{\"participantID\": \"\$(p.id?.uuidString ?? \"\")\"}",
            riskScore: 0.1
        )
        try await run.save(on: req.db)
        return p
    }

    app.post("identity", "upload") { req async throws -> HTTPStatus in
        let input = try req.content.decode(UploadIdentityRequest.self)
        guard let participant = try await Participant.find(input.participantID, on: req.db) else {
            throw Abort(.notFound)
        }
        participant.status = input.documentURL.isEmpty ? "needs_review" : "identity_submitted"
        try await participant.save(on: req.db)
        return .accepted
    }

    app.post("workflows", "approve") { req async throws -> HTTPStatus in
        let input = try req.content.decode(ApproveRequest.self)
        guard let workflow = try await WorkflowRun.find(input.workflowID, on: req.db) else {
            throw Abort(.notFound)
        }
    }
}

```

```

        workflow.status = "approved"
        try await workflow.save(on: req.db)
        return .ok
    }

    app.get("health") { _ async in
        "ok"
    }
}

```

implementation-pack/Sources/App/WCSModels.swift

```

import Fluent
import Vapor

struct CreateParticipant: Content {
    let email: String
    let fullName: String
}

struct UploadIdentityRequest: Content {
    let participantID: UUID
    let documentURL: String
}

struct ApproveRequest: Content {
    let workflowID: UUID
    let approvedBy: String
}

final class Participant: Model, Content, @unchecked Sendable {
    static let schema = "participants"

    @ID(key: .id)
    var id: UUID?

    @Field(key: "email")
    var email: String

    @Field(key: "full_name")
    var fullName: String

    @Field(key: "status")
    var status: String

    @Timestamp(key: "created_at", on: .create)
    var createdAt: Date?

    init() {}

    init(id: UUID? = nil, email: String, fullName: String, status: String = "pending") {
        self.id = id
        self.email = email
        self.fullName = fullName
        self.status = status
    }
}

final class WorkflowRun: Model, Content, @unchecked Sendable {
    static let schema = "workflow_runs"

    @ID(key: .id)
    var id: UUID?

    @Field(key: "workflow_type")
    var workflowType: String

    @Field(key: "status")
    var status: String

    @Field(key: "payload")
    var payload: String

    @Field(key: "risk_score")
    var riskScore: Double

    @Timestamp(key: "created_at", on: .create)

```

```

    var createdAt: Date?

    init() {}

    init(workflowType: String, status: String = "queued", payload: String = "{}", riskScore: Double = 0) {
        self.workflowType = workflowType
        self.status = status
        self.payload = payload
        self.riskScore = riskScore
    }
}

```

implementation-pack/Sources/App/Migrations.swift

```

import Fluent

struct CreateParticipantMigration: AsyncMigration {
    func prepare(on db: Database) async throws {
        try await db.schema(Participant.schema)
            .id()
            .field("email", .string, .required)
            .field("full_name", .string, .required)
            .field("status", .string, .required)
            .field("created_at", .datetime)
            .unique(on: "email")
            .create()
    }

    func revert(on db: Database) async throws {
        try await db.schema(Participant.schema).delete()
    }
}

struct CreateWorkflowRunMigration: AsyncMigration {
    func prepare(on db: Database) async throws {
        try await db.schema(WorkflowRun.schema)
            .id()
            .field("workflow_type", .string, .required)
            .field("status", .string, .required)
            .field("payload", .string, .required)
            .field("risk_score", .double, .required)
            .field("created_at", .datetime)
            .create()
    }

    func revert(on db: Database) async throws {
        try await db.schema(WorkflowRun.schema).delete()
    }
}

```